University of Tabuk
Faculty of Computers & Information Technology
Department of Computer Engineering

MICROPROCESSORS & EMBEDDED SYSTEMS LAB
CEN-412

Project (2). Parking Sensor for Garage Side — 10 points

1. Project description

This project is a Distance Measurement and Display System that uses an Ultrasonic Sensor (HC-SR04) to measure the distance of an object and displays the result in centimeters on a 4-digit 7-segment display controlled by a 74HC595 shift register.

The system also includes LED indicators (green, yellow, and red) and a buzzer to provide visual and audible warnings depending on how close an object is.

When the ultrasonic sensor sends out sound waves, they bounce back from the nearest object, and the system calculates the time it takes for the echo to return. This time is converted into distance using the speed of sound.

The calculated distance is then broken down into individual digits and shown on the 7-segment display using a multiplexing technique to make all four digits appear simultaneously.

If the distance is:

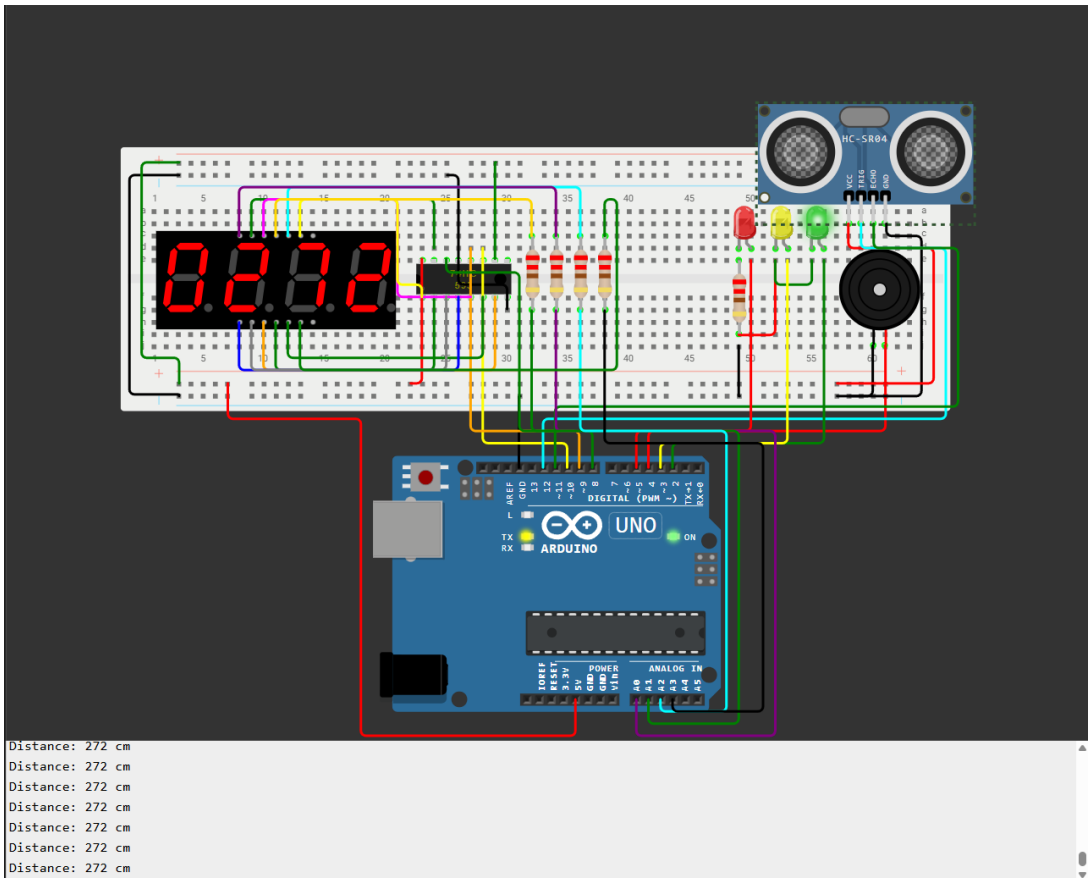
More than 30 cm: The green LED lights up (safe distance).

Between 10 and 30 cm → The yellow LED blinks and the buzzer beeps intermittently (warning).

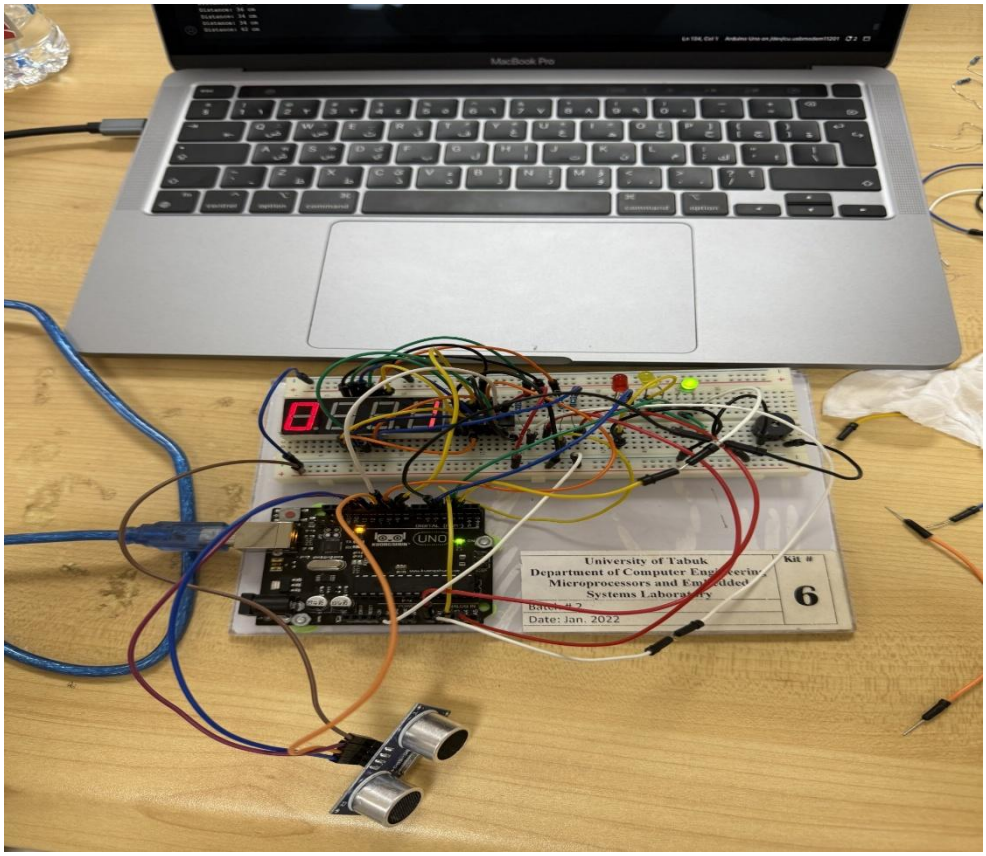
Less than 10 cm: The red LED lights up and the buzzer emits a continuous tone (danger zone).

This project can be used in parking sensor systems, robotic obstacle detection, or any application that requires real-time distance monitoring and alerting.

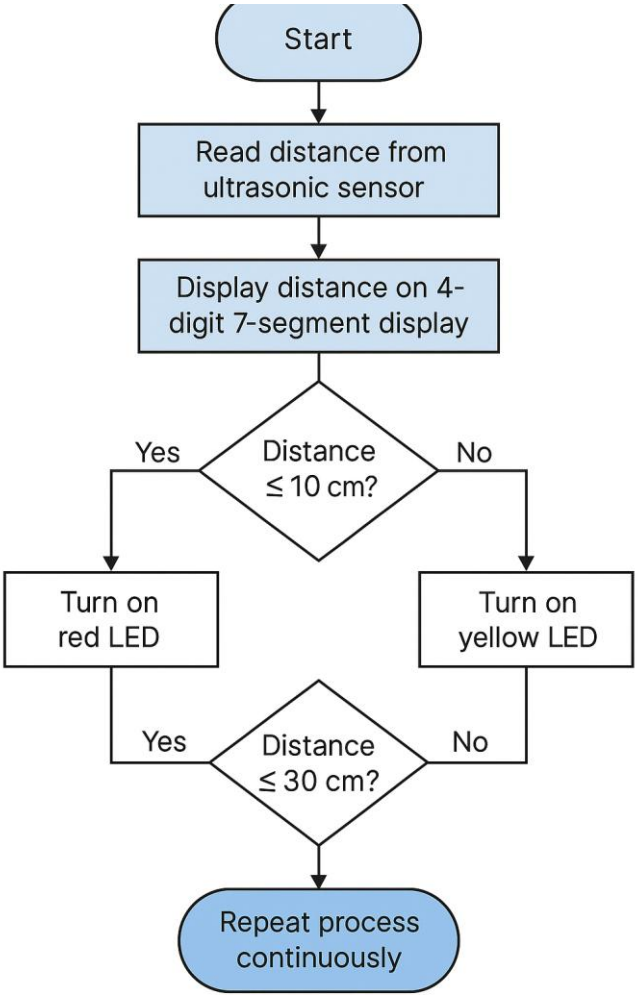
2. Project schematic Diagram



In Lab:



3. Algorithm and Program source code




```
#define TRIG_PIN 12
#define ECHO_PIN 11
#define BUZZER   5

#define LED_G 2
#define LED_Y 3
#define LED_R 4

#define SER_PIN   8
#define RCLK_PIN  9
#define SRCLK_PIN 10

#define DIG1 A0
#define DIG2 A1
#define DIG3 A2
#define DIG4 A3
```

```
unsigned char seg[] =
{0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,0x7f,0x6f,0x77,0x7c
,0x39,0x5e,0x79,0x71,0x00};

long duration;
int distanceCm;

void sendData(byte data) {

  digitalWrite(RCLK_PIN, LOW);
  shiftOut(SER_PIN, SRCLK_PIN, MSBFIRST, data);
  digitalWrite(RCLK_PIN, HIGH);
}

void showDigit(char digit, char value) {

  digitalWrite(DIG1, HIGH);
  digitalWrite(DIG2, HIGH);
  digitalWrite(DIG3, HIGH);
  digitalWrite(DIG4, HIGH);

  sendData(seg[value]);

  switch (digit) {
    case 1: digitalWrite(DIG1, LOW); break;
    case 2: digitalWrite(DIG2, LOW); break;
    case 3: digitalWrite(DIG3, LOW); break;
    case 4: digitalWrite(DIG4, LOW); break;
  }

  delay(3);
}

void showNumber(int num) {
  int d1 = (num / 1000) % 10;
  int d2 = (num / 100) % 10;
  int d3 = (num / 10)  % 10;
  int d4 = num % 10;
```

```
  for (int i = 0; i < 20; i++) {
    showDigit(4, d4);
    showDigit(3, d3);
    showDigit(2, d2);
    showDigit(1, d1);
  }

}


void readDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  duration = pulseIn(ECHO_PIN, HIGH, 30000UL);
  if (duration == 0) distanceCm = 400;
  else distanceCm = duration * 0.034 / 2;

  if (distanceCm > 400) distanceCm = 400;
}

void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(BUZZER, OUTPUT);
  pinMode(LED_G, OUTPUT);
  pinMode(LED_Y, OUTPUT);
  pinMode(LED_R, OUTPUT);

  pinMode(SER_PIN, OUTPUT);
  pinMode(RCLK_PIN, OUTPUT);
  pinMode(SRCLK_PIN, OUTPUT);
  pinMode(DIG1, OUTPUT);
  pinMode(DIG2, OUTPUT);
  pinMode(DIG3, OUTPUT);
  pinMode(DIG4, OUTPUT);
```

```
  Serial.begin(9600);
}

void loop() {
  readDistance();
  showNumber(distanceCm);

  Serial.print("Distance: ");
  Serial.print(distanceCm);
  Serial.println(" cm");

  if (distanceCm <= 10) {
    digitalWrite(LED_R, HIGH);
    digitalWrite(LED_Y, LOW);
    digitalWrite(LED_G, LOW);
    tone(BUZZER, 1000);
  }
  else if (distanceCm <= 30) {
    digitalWrite(LED_R, LOW);
    digitalWrite(LED_Y, HIGH);
    digitalWrite(LED_G, LOW);
    tone(BUZZER, 1000);
    delay(100);
    noTone(BUZZER);
    delay(100);
  }
  else {
    digitalWrite(LED_R, LOW);
    digitalWrite(LED_Y, LOW);
    digitalWrite(LED_G, HIGH);
    noTone(BUZZER);
  }

  delay(50);
}
```

4. Program description

At the beginning of the program, all the pins used in the project are defined

for the ultrasonic sensor (TRIG and ECHO), the LEDs and buzzer,

and the 4-digit 7-segment display, which is controlled by the 74HC595 shift register.

An array is also created that stores the binary patterns of digits (0–9) to display them correctly on the 7-segment display.

- sendData() Function

This function sends data from the Arduino to the 74HC595 shift register.

The register controls which segments of the display should light up to form the correct number.

In short, this function "tells the display what number to show."

- showDigit() Function

This function displays one digit on a specific position of the 4-digit display.

It first turns off all digits, then activates only one digit and sends the data pattern for the number to be shown.

A very short delay keeps the number visible before switching to the next digit.

This process is part of the multiplexing technique that makes all four digits appear at the same time.

- showNumber() Function

This function receives the full number (for example, the measured distance of 76 cm).

It separates the number into individual digits—thousands, hundreds, tens, and ones.

Then it calls showDigit() repeatedly for each digit in a quick loop.

so the entire number appears stable and clear on the 4-digit display.

- readDistance() Function

This function reads the distance from the HC-SR04 ultrasonic sensor.

It sends a short trigger pulse, waits for the echo, and measures the time it takes for the sound wave to return.

The distance in centimeters is calculated using the formula based on the speed of sound.

and the result is stored in the variable distanceCm.

- setup() Function

This function runs once at the start of the program.

It sets all pins as inputs or outputs and starts the Serial Monitor for debugging.

Basically, it prepares all the components before the main loop starts running.

- loop() Function

This is the main part of the program that runs continuously.

In each cycle:

It measures the distance by calling readDistance().

It displays the measured distance using showNumber().

It prints the distance value on the Serial Monitor.

It checks the distance range and activates the LEDs and buzzer accordingly:

Less than 10 cm → Red LED ON, buzzer sounds continuously.

Between 10 and 30 cm → Yellow LED blinks, buzzer beeps intermittently.

Greater than 30 cm → Green LED ON, buzzer OFF.

The loop repeats constantly to update the distance in real time and provide instant visual and audio feedback.